🛠️ Final Capstone Project: Deploy Your Customer Churn Prediction Model

You've trained, tuned, and evaluated your churn prediction model — now it's time to deploy it!

In this final capstone project, your job is to **deploy your churn model** as a real-time API using **Flask**, and create a batch pipeline that scores customers automatically. You will also track the model's behavior and propose a basic monitoring and maintenance strategy.

You can download your data (`gold_churn_data.csv`) from here and transform it using the provided preprocessing code.

```
# Creating X and y
X = df.drop("Churn", axis=1)
y = df["Churn"]

# Step 1: Drop the 'customerID' column
X = X.drop(columns=['customerID'])
# Step 2: Convert 'TotalCharges' to numeric (handles spaces or
non-numeric values)
X['TotalCharges'] = pd.to_numeric(X['TotalCharges'],
errors='coerce')
# Step 3: Convert target column 'y' to binary values
y = y.map({'Yes': 1, 'No': 0})
# Step 4: Identify column types
categorical_cols = X.select_dtypes(include=
['object']).columns.tolist()
numerical_cols = X.select_dtypes(include=['int64',
'float64']).columns.tolist()
# Step 5: Define preprocessing pipeline (no model yet)
preprocessor = ColumnTransformer(transformers=[
    ('num', SimpleImputer(strategy='mean'), numerical_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse=False),
categorical_cols)
])
# Step 6: Apply the transformation to the data
X_cleaned = preprocessor.fit_transform(X)
```

✅ What You'll Build

You will Train a ML model using this data and store the same along with your pipeline.

Your solution must include:

1. 🚀 Real-Time Inference API (Flask)

- Structure your code as a **Python module** named `app/` :

  - Include an empty `__init__.py` file in `app/` so it can be run as a module.
- Create a Flask app in `app/main.py` with a **`/predict`** endpoint.
- Accept POST requests with raw customer JSON data, for example:

```
{
  "customer": {
    "gender": "Female",
    "SeniorCitizen": 0,
    "Partner": "Yes",
    "tenure": 12,
    ...
  }
}
```

- Internally:
  1. Convert this JSON into a single-row `pandas.DataFrame`.
  2. Apply the **same** preprocessing pipeline (`transformer.pkl`) used during training.
  3. Pass the transformed array to your model (`model.pkl`).
- Return JSON:

```
{
  "churn_probability": 0.83,
  "churn_prediction": "Yes"
}
```

---

2. 🖥️ How to Run the API

From the **project root**, start your server on **port 8000** by running as a module:

```
cd customer-churn-api
python -m app.main
```

- Make sure `app/__init__.py` exists so Python treats `app` as a package.
- Your Flask app must listen on `localhost:8000`.

---

3. 🧪 Batch Scoring Script

Implement `batch.py` to:

1. Accept `--input test_data/all_customers.csv`.
2. Send each row as JSON to `http://localhost:8000/predict`.
3. Collect responses and write `scored_customers.csv`.
4. Log total requests, failures, and average probability into `logs/batch_log.txt`.

Example:

```
python batch.py --input test_data/all_customers.csv
```

🔍 Logging & Monitoring

While running `batch.py`:

- Log **total number of requests**.
- Log **failed predictions** (exceptions or non-200 responses).
- Log **average churn probability**.

Use Python's `logging` module; ensure a `logs/` directory exists.

📊 Maintenance Plan (`README.md`)

In this same `README.md`, include a concise maintenance plan:

- 🧠 **Retraining**: when and how you will retrain the model.
- 📉 **Drift detection**: how you will monitor for data or model drift.
- 🗃️ **Versioning**: strategy for model and pipeline updates.

Keep each section under 200 words.

📦 Submission Structure

Your GitHub repo must follow this exact structure:

```
customer-churn-api/
│
├── app/
│   ├── __init__.py          # empty file, makes 'app' a
module
│   ├── main.py              # Flask application, run via
python -m app.main
│   ├── model.pkl            # Trained churn model
│   ├── transformer.pkl      # Preprocessing pipeline
│   └── utils.py             # Helper functions
│
├── test_data/
│   ├── sample_input.json    # JSON input for /predict
endpoint
│   └── all_customers.csv    # CSV input for batch scoring
│
├── batch.py                 # Batch scoring script
├── requirements.txt         # Python dependencies
├── README.md                # This file (updated
instructions + maintenance plan)
└── .gitignore
```

> **Do not** rename any files or directories.

🧪 Auto-Grading

We will automatically:

1. Clone your repo.

2. Install `requirements.txt`.
3. Run `python -m app.main`.
4. POST `test_data/sample_input.json` to `http://localhost:8000/predict`.
5. Run `python batch.py --input test_data/all_customers.csv`.
6. Validate `scored_customers.csv` and check logs.

Good luck! 🚀